



LISTA DE EXERCÍCIOS - SISTEMAS INTELIGENTES APLICADOS

Lista de Exercícios 4

Árvore de Decisão

DOCENTE: Thiago França Naves

DATA: 30/09/2026

ALUNO: Mayara Rodrigues Pereira e Leonardo José Reis Pinto

As entropias, os ganhos e a árvore das questões 1 e 3 foram calculados pelo script `solucao.py` (Apêndice A), que implementa o ID3. A questão 4 foi feita no Weka 3.8.6, com o ID3 do pacote `simpleEducationalLearningSchemes` e validação cruzada de 10 dobras.

Questão 1

Considere a tabela abaixo com atributos e entradas para um problema de classificação (14 exemplos com aparência, temperatura, umidade, ventando e jogar). a) Calcule a entropia inicial. b) De acordo com o ganho de informação qual seria a montagem dessa árvore de decisão? c) Qual a taxa de acerto da árvore de decisão para o seguinte conjunto de teste.

Resposta. a) Dos 14 exemplos, 9 são “sim” e 5 são “não”:

$$H(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0,940 \text{ bits.}$$

b) O ID3 escolhe, em cada nó, o atributo com maior ganho $G(S, A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$. Temperatura e umidade são numéricas, e o ID3 só trabalha com valores discretos. Como o conjunto de teste já vem com as categorias (Cool, Mild, Hot e Normal, High), dividimos em faixas: temperatura fria abaixo de 70, amena de 70 a 79 e quente de 80 para cima, e umidade normal até 80 e alta acima de 80. Essas faixas reproduzem as categorias da versão nominal dessa mesma base, que vem com o Weka.

Na raiz:

Atributo	Divisão (sim / não)	Ganho
Aparência	sol 2/3, nublado 4/0, chuva 3/2	0,247
Umidade	normal 6/1, alta 3/4	0,152
Ventando	falso 6/2, verdadeiro 3/3	0,048
Temperatura	fria 3/1, amena 4/2, quente 2/2	0,029

Para a aparência, por exemplo: $0,940 - \frac{5}{14} \cdot 0,971 - \frac{4}{14} \cdot 0 - \frac{5}{14} \cdot 0,971 = 0,247$. Ela vira a raiz. O ramo “nublado” tem só “sim” e já é folha. No ramo “sol” (5 exemplos, 2 sim e 3 não) o melhor atributo é a umidade, com ganho 0,971, que separa tudo. No ramo “chuva” é o ventando, também com ganho 0,971.



Figura 1. Árvore montada pelo ganho de informação.

O corte da umidade em 80 não é o único possível: com o corte em 75 a árvore sai igual. No ramo “sol” as umidades são 70 e 70 contra 85, 90 e 95, então qualquer corte nesse intervalo separa as classes.

c) Passando os cinco exemplos de teste pela árvore:

Outlook	Temp	Humidity	Windy	Real	Previsto	
Rainy	Cool	Normal	True	Yes	No	erro
Rainy	Mild	Normal	False	Yes	Yes	
Overcast	Hot	High	False	Yes	Yes	
Overcast	Hot	High	True	No	Yes	erro
Sunny	Cool	High	True	No	No	

A taxa de acerto é de 3 em 5, ou **60%**. Os dois erros contrariam padrões que a base de treino não tinha: um dia de chuva com vento em que se jogou e um dia nublado em que não se jogou (no treino, todos os dias nublados eram “sim”).

Questão 2

Desenhe árvores de decisão que representem os seguintes conceitos (sendo A , B , C e D variáveis booleanas): a) $A \wedge \neg B$; b) $A \vee (B \wedge C)$; c) $A \neq B$; d) $(A \wedge B) \vee (C \wedge D)$.

Resposta. Em cada nó testamos uma variável. O ramo da esquerda é verdadeiro (V) e o da direita é falso (F).

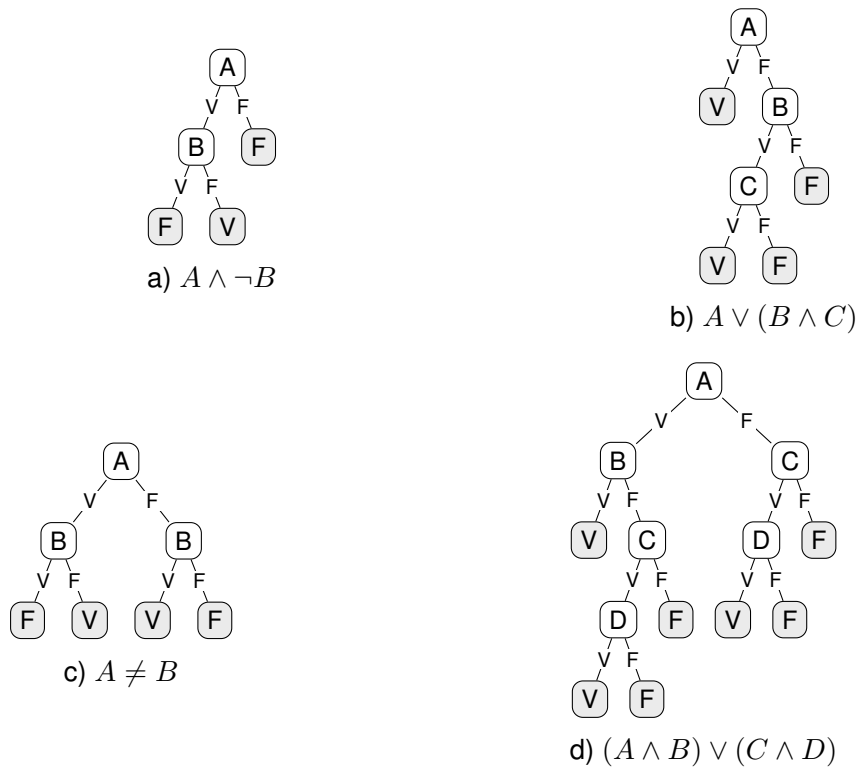


Figura 2. Árvores dos quatro conceitos.

No item c o teste de B aparece nos dois ramos, porque o ou exclusivo só é decidido olhando as duas variáveis juntas. No item d a subárvore de $C \wedge D$ tem que ser repetida em dois lugares: quando A é falso e quando A é verdadeiro mas B é falso. Uma árvore não consegue reaproveitar uma subárvore, e por isso disjunções de conjunções deixam a árvore maior do que a fórmula.

Questão 3

Considere o conjunto de treino (6 exemplos com A1 e A2). a) Calcule a informação esperada ou entropia deste conjunto de exemplos e o ganho de informação relativo ao atributo A2. A2 poderá ser raiz da árvore de decisão obtida pelo algoritmo ID3? b) Apresente a árvore de decisão completa que seria produzida pelo algoritmo ID3.

Resposta. a) Há 3 exemplos positivos e 3 negativos, então $H(S) = 1$ bit, a maior entropia possível para duas classes.

A2 = alto tem os exemplos 1, 2, 5 e 6, com 2 positivos e 2 negativos, entropia 1. A2 = baixo tem os exemplos 3 e 4, com 1 de cada, entropia 1. Logo

$$G(S, A2) = 1 - \frac{4}{6} \cdot 1 - \frac{2}{6} \cdot 1 = 0.$$

Para A1: escuro tem 1, 4 e 5 (1 positivo, 2 negativos) e claro tem 2, 3 e 6 (2 positivos, 1 negativo), os dois com entropia 0,918, e $G(S, A1) = 1 - 0,918 = 0,082$.

A2 **não** pode ser a raiz, porque o ID3 escolhe o atributo de maior ganho e A1 tem ganho maior (0,082 contra 0).

b) Com A1 na raiz, sobra só A2 para cada ramo, e em cada ramo ele separa as classes por completo (ganho 0,918):

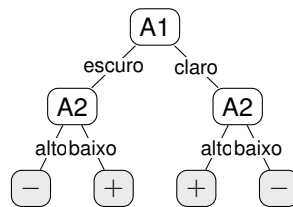


Figura 3. Árvore completa do ID3.

Esse caso mostra uma limitação do critério guloso. A classe é um ou exclusivo de A1 e A2 (com exceção de um exemplo), e cada atributo sozinho diz muito pouco. A2 tem ganho zero na raiz, mas é ele que resolve a árvore no nível seguinte.

Questão 4

Descreva como funciona o algoritmo de árvore de decisão ID3. Em seguida, utilize a ferramenta Weka e faça testes com o algoritmo ID3 e algoritmo J48, utilizando as bases Weather Nominal e Diabetes. Descreva as diferenças dos algoritmos nas duas bases em relação aos valores de acurácia, número de nós folhas, tamanho da árvore gerada e matriz de confusão. Porque o algoritmo ID3 não processa a base Diabetes?

Resposta. O ID3 monta a árvore de cima para baixo. Na raiz, calcula o ganho de informação de cada atributo e escolhe o maior. Cria um ramo para cada valor desse atributo e repete o processo em cada ramo, com os exemplos que chegaram nele e sem o atributo já usado. Um ramo vira folha quando todos os exemplos são da mesma classe ou quando os atributos acabam, e aí recebe a classe da maioria. Ele não volta atrás nas escolhas, não poda a árvore e só aceita atributos nominais. O J48 é a versão do C4.5 no Weka. Ele usa a razão de ganho, trata atributos numéricos com cortes do tipo “atributo $\leq$ limiar”, aceita valores faltando e poda a árvore no final.

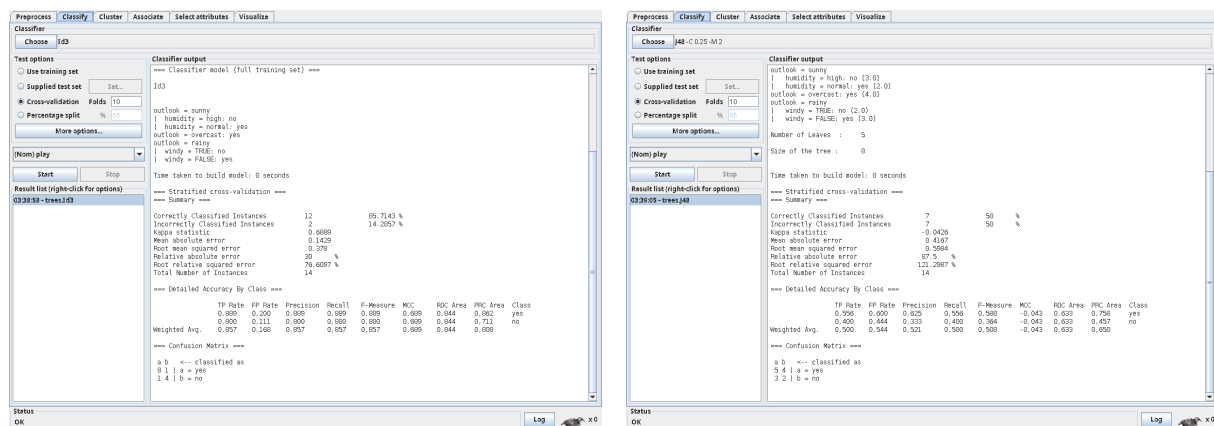


Figura 4. ID3 (esquerda) e J48 (direita) na Weather Nominal.

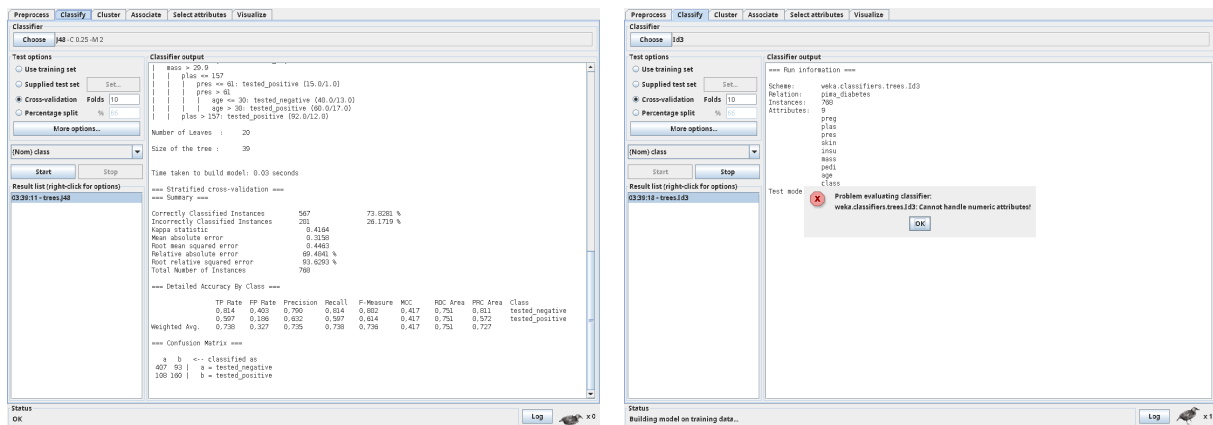


Figura 5. J48 na Diabetes (esquerda) e o erro do ID3 na mesma base (direita).

Tabela 1. Resultados com validação cruzada de 10 dobras.

Base	Algoritmo	Acurácia	Folhas	Tamanho	Matriz de confusão
Weather	ID3	85,71%	5	8	[8 1; 1 4]
Weather	J48	50,00%	5	8	[5 4; 3 2]
Diabetes	ID3	não roda: “Cannot handle numeric attributes”			
Diabetes	J48	73,83%	20	39	[407 93; 108 160]

O ID3 do Weka não mostra o número de folhas, então contamos na árvore impressa.

Na Weather os dois algoritmos montam a mesma árvore com a base inteira, igual à da questão 1: aparência na raiz, umidade no ramo “sunny” e ventando no ramo “rainy”, com 5 folhas e 8 nós. A diferença está na validação cruzada. Para entender, refizemos as 10 dobras do Weka (mesma semente) e olhamos as árvores de cada uma. O ID3 sempre começa pela aparência e erra só 2 exemplos. O J48, com 12 ou 13 exemplos de treino, colocou a *umidade* na raiz em 4 das 10 dobras. Ele escolhe pela razão de ganho, que divide o ganho pela entropia da própria divisão, e isso favorece a umidade, com 2 valores, contra a aparência, com 3. Além disso, a poda (no mínimo 2 exemplos por folha e fator de confiança 0,25) juntou ramos pequenos: numa dobra a árvore ficou só com a umidade, 2 folhas, e errou um dia nublado, que nessa base é sempre “yes”. Como essa base não tem ruído, a árvore completa do ID3 generaliza melhor: 12 acertos contra 7. Com 14 exemplos cada instância vale 7 pontos percentuais, então essa diferença também tem muito de acaso.

Na Diabetes só o J48 funciona. A árvore ficou bem maior, com 20 folhas e 39 nós, porque ele precisa fazer vários cortes nos mesmos atributos numéricos (glicose, IMC, idade) em alturas diferentes. A acurácia foi de 73,83%. A matriz mostra que o erro maior está nos diabéticos: 108 dos 268 foram classificados como saudáveis, contra 93 erros entre os 500 saudáveis.

O ID3 não processa a Diabetes porque todos os atributos dela são numéricos contínuos, e o ID3 cria um ramo para cada valor distinto do atributo. Com valores contínuos haveria quase um ramo por paciente, cada um com um ou dois exemplos. O ganho de informação desses atributos seria artificialmente alto e a árvore não generalizaria nada. Por isso a implementação do Weka recusa a base com a mensagem da figura. Para usar o ID3 ali seria preciso discretizar antes, por exemplo com o filtro Discretize.

Apêndice A. Código

python solucao.py

```
# Lista 4 - Árvore de Decisão
# Sistemas Inteligentes Aplicados - UTFPR
# Uso: python solucao.py

from collections import Counter
from math import log2

def entropia(classes):
    total = len(classes)
    h = 0
    for n in Counter(classes).values():
        p = n / total
        h -= p * log2(p)
    return h

def ganho(base, attr, alvo):
    # base: lista de dicionários
    h = entropia([x[alvo] for x in base])
    resto = 0
    for v in set(x[attr] for x in base):
        sub = [x[alvo] for x in base if x[attr] == v]
        resto += len(sub) / len(base) * entropia(sub)
    return h - resto

def id3(base, attrs, alvo, nivel=0, mostrar=True):
    classes = [x[alvo] for x in base]
    if len(set(classes)) == 1 or not attrs:
        return Counter(classes).most_common(1)[0][0]
    ganhos = {a: ganho(base, a, alvo) for a in attrs}
    if mostrar:
        txt = " ".join(f"{a}={g:.3f}" for a, g in ganhos.items())
        print(" " + " " * nivel + f"[{len(base)} exemplos] ganhos: {txt}")
    melhor = max(attrs, key=lambda a: ganhos[a])
    arvore = {melhor: {}}
    for v in sorted(set(x[melhor] for x in base)):
        sub = [x for x in base if x[melhor] == v]
        resto = [a for a in attrs if a != melhor]
        arvore[melhor][v] = id3(sub, resto, alvo, nivel + 1, mostrar)
    return arvore

def imprimir(arvore, nivel=0):
    if not isinstance(arvore, dict):
        return
    attr = list(arvore)[0]
    for v, sub in arvore[attr].items():
        if isinstance(sub, dict):
            print(" " + "|" + " " * nivel + f"{attr} = {v}")
            imprimir(sub, nivel + 1)
        else:
            print(" " + "|" + " " * nivel + f"{attr} = {v}: {sub}")

def classificar(arvore, x):
    while isinstance(arvore, dict):
        attr = list(arvore)[0]
        arvore = arvore[attr].get(x[attr])
    return arvore

# questao 1
print("[1] base do jogo com temperatura e umidade numéricas")
tabela = """sol 85 85 falso nao
sol 80 90 verdadeiro nao
nublado 83 86 falso sim
chuva 70 96 falso sim
```

```

chuva 68 80 falso sim
chuva 65 70 verdadeiro nao
nublado 64 65 verdadeiro sim
sol 72 95 falso nao
sol 69 70 falso sim
chuva 75 80 falso sim
sol 75 70 verdadeiro sim
nublado 72 90 verdadeiro sim
nublado 81 75 falso sim
chuva 71 91 verdadeiro nao"""

def faixa_temp(t):
    if t < 70:
        return "fria"
    if t < 80:
        return "amena"
    return "quente"

def faixa_umid(u):
    return "normal" if u <= 80 else "alta"

jogo = []
for linha in tabela.split("\n"):
    ap, t, u, v, c = linha.split()
    jogo.append({"aparencia": ap, "temperatura": faixa_temp(int(t)),
                "umidade": faixa_umid(int(u)), "ventando": v, "jogar": c})

print(f" a) entropia inicial = {entropia([x['jogar'] for x in jogo]):.4f}")
attrs = ["aparencia", "temperatura", "umidade", "ventando"]
print(" b) construção pelo ID3:")
arvore = id3(jogo, attrs, "jogar")
imprimir(arvore)

# confere se a árvore muda com outro corte na umidade
for corte in [75]:
    base = [dict(x) for x in jogo]
    for x, linha in zip(base, tabela.split("\n")):
        x["umidade"] = "normal" if int(linha.split()[2]) <= corte else "alta"
    print(f" com umidade normal <= {corte}: ",
          id3(base, attrs, "jogar", mostrar=False) == arvore)

teste = [("chuva", "fria", "normal", "verdadeiro", "sim"),
         ("chuva", "amena", "normal", "falso", "sim"),
         ("nublado", "quente", "alta", "falso", "sim"),
         ("nublado", "quente", "alta", "verdadeiro", "nao"),
         ("sol", "fria", "alta", "verdadeiro", "nao")]

acertos = 0
print(" c) conjunto de teste:")
for ap, t, u, v, c in teste:
    x = {"aparencia": ap, "temperatura": t, "umidade": u, "ventando": v}
    prev = classificar(arvore, x)
    acertos += prev == c
    print(f" {ap}, {t}, {u}, {v}: real={c} previsto={prev}")
print(f" taxa de acerto = {acertos}/5 = {100 * acertos / 5:.0f}%")

# questao 3
print("\n[3] A1 e A2")
base3 = [{"A1": "escuro", "A2": "alto", "c": "-"},
         {"A1": "claro", "A2": "alto", "c": "+"},
         {"A1": "claro", "A2": "baixo", "c": "-"},
         {"A1": "escuro", "A2": "baixo", "c": "+"},
         {"A1": "escuro", "A2": "alto", "c": "-"},
         {"A1": "claro", "A2": "alto", "c": "+"}]

print(f" entropia = {entropia([x['c'] for x in base3]):.4f}")
print(f" ganho(A1) = {ganho(base3, 'A1', 'c'):.4f}")
print(f" ganho(A2) = {ganho(base3, 'A2', 'c'):.4f}")
arvore3 = id3(base3, ["A1", "A2"], "c")
imprimir(arvore3)

```